

TALLER DE PYTHON

Ciencia de Datos con Canciones de TikTok 2020

Nivel Intermedio | Primer Semestre

3 Modulos
Limpieza,
Estandarizacion
y Graficas

292 Canciones
Dataset real de
TikTok 2020

4 Graficas
Barras, Histograma
Scatter y Pie

Pandas + NumPy
+ Matplotlib
Todo lo que necesitas

En este taller vas a aprender a analizar un dataset REAL de TikTok usando Python. Aprenderemos a limpiar datos sucios, estandarizarlos y crear graficas impresionantes. Todo paso a paso, con explicaciones sencillas. ¡Lets go!

Vista previa del Dataset

track_name	artist_name	track_pop	danceability	energy	tempo
Say So	Doja Cat	80	0.787	0.673	110.9
Blinding Lights	The Weeknd	90	0.514	0.730	171.0
Savage	Megan Thee Stallion	70	0.843	0.741	168.9
cardigan	Taylor Swift	79	0.613	0.581	130.0
Physical	Dua Lipa	75	0.647	0.844	146.9

El dataset tiene **292 canciones** con **18 columnas** que incluyen: nombre del artista, popularidad, bailabilidad, energia, tempo, acusticidad, y mucho mas.

MODULO 1 — Limpieza de Datos con Pandas

Antes de analizar cualquier dataset, necesitamos LIMPIARLO. Los datos del mundo real vienen sucios: con valores que faltan, columnas innecesarias, tipos de datos incorrectos... En este modulo aprenderemos a detectar y solucionar todos esos problemas.

Paso 1 Cargar el dataset e inspeccionar

Lo primero es importar las librerias y cargar el archivo CSV. Piensalo como abrir el Excel pero desde Python. Luego le "pedimos" al dataset que nos cuente como esta:

```
# Importamos las librerias (como abrir las apps que necesitamos)
import pandas as pd
import numpy as np

# Cargamos el dataset (como abrir un archivo de Excel)
df = pd.read_csv("TikTok_songs_2020.csv")

# Cuantas filas y columnas tiene?
print("Forma del dataset:", df.shape)

# Ver los primeros 5 registros
print(df.head())

# Ver informacion general de cada columna
df.info()
```

[INFO] df.shape te da (filas, columnas). Nuestro dataset tiene 292 canciones y 18 columnas. ¡Como una tabla de Excel con 292 filas!

Paso 2 Detectar valores nulos (datos que faltan)

Un valor nulo es como una celda vacia en Excel. Si calculamos promedios con celdas vacias, obtenemos resultados erroneos. Pandas tiene funciones magicas para encontrarlos:

```
# Contar cuantos valores nulos tiene cada columna
print(df.isnull().sum())

# Ver el porcentaje de nulos por columna
porcentaje_nulos = (df.isnull().sum() / len(df)) * 100
print(porcentaje_nulos)

# Visualizar rapidamente si HAY nulos en el dataset
print("Hay nulos?", df.isnull().values.any())
```

Salida esperada:

```
track_name 0
artist_name 0
artist_pop 0
track_pop 0
danceability 0
... (todas en 0)
```

Hay nulos? **False** – Nuestro dataset esta limpio!

Paso 3 Eliminar duplicados y columnas innecesarias

Imagina que la misma cancion aparece dos veces en la lista. Eso arruinaria nuestros calculos. Tambien quitaremos columnas que no usaremos en nuestro analisis:

```
# Ver si hay canciones duplicadas
print("Duplicados:", df.duplicated().sum())

# Eliminar duplicados (si los hay)
df = df.drop_duplicates()

# Eliminar columnas que no necesitamos en este analisis
# (time_signature y key son muy tecnicas para comenzar)
columnas_a_quitar = ["time_signature", "key"]
df = df.drop(columns=columnas_a_quitar)

# Verificar el nuevo tamaño del dataset
print("Nuevo tamaño:", df.shape)

# Ver las columnas que quedan
print("Columnas:", df.columns.tolist())
```

[OJO] drop_duplicates() elimina filas repetidas. drop(columns=...) elimina columnas. ¡Siempre guarda el resultado en df para que los cambios persistan!

Paso 4 Convertir duracion de ms a minutos

La columna duration_ms tiene la duracion en milisegundos. ¡Eso es difícil de entender! Vamos a crear una nueva columna con la duracion en minutos, que es mas intuitivo:

```
# Crear nueva columna: duracion en minutos
# (dividimos entre 1000 para pasar a segundos, luego entre 60 para minutos)
df["duration_min"] = df["duration_ms"] / 1000 / 60

# Redondear a 2 decimales para que se vea limpio
df["duration_min"] = df["duration_min"].round(2)

# Ver las estadísticas de la nueva columna
print(df["duration_min"].describe())

# Cual es la cancion mas corta y mas larga?
print("Mas corta:", df.loc[df["duration_min"].idxmin(), "track_name"])
print("Mas larga:", df.loc[df["duration_min"].idxmax(), "track_name"])
```

Resultado:

count 292.000

mean 2.81 min

min 1.00 min

max 7.79 min

Mas corta: Hungry Hippo (Tierra Whack)

Mas larga: Let It Happen (Tame Impala)

MODULO 2 — Estandarizacion con Pandas y NumPy

La estandarizacion es el proceso de transformar los datos para que sean comparables entre si. Por ejemplo, "danceability" va de 0 a 1, pero "tempo" va de 70 a 200+. ¡Son escalas completamente distintas! Normalizar los datos permite compararlos y usarlos en modelos de machine learning.

Paso 1 Estadísticas descriptivas

Antes de estandarizar, necesitamos conocer los datos. `describe()` es como pedirle a un amigo que te resuma el dataset:

```
# Estadísticas de todas las columnas numericas de un vistazo
print(df.describe().round(2))

# Calcular estadísticas específicas con NumPy
columnas_audio = ["danceability", "energy", "valence", "tempo"]

for col in columnas_audio:
    print(f"\n--- {col} ---")
    print(f" Media: {np.mean(df[col]):.3f}")
    print(f" Mediana: {np.median(df[col]):.3f}")
    print(f" Desv Std: {np.std(df[col]):.3f}")
    print(f" Min: {np.min(df[col]):.3f}")
    print(f" Max: {np.max(df[col]):.3f}")
```

[INFO] `np.mean()` calcula el promedio, `np.median()` el valor del centro, y `np.std()` que tan "dispersos" estan los datos. ¡NumPy hace estos calculos MUCHO mas rapido que Python puro!

Paso 2 Normalizacion Min-Max (escala 0 a 1)

La normalizacion Min-Max convierte todos los valores a una escala de 0 a 1. Si el tempo mas bajo es 72 y el mas alto es 210, un tempo de 141 quedaria justo en el centro (0.5). Asi podemos comparar tempo con danceability directamente:

```
# Formula de normalizacion Min-Max:
# valor_norm = (valor - minimo) / (maximo - minimo)

# Aplicar normalizacion a columnas de audio
columnas_a_normalizar = ["danceability", "energy", "valence",
                        "tempo", "acousticness", "speechiness"]

df_normalizado = df.copy() # Hacemos una copia para no perder los originales

for col in columnas_a_normalizar:
    minimo = df[col].min()
    maximo = df[col].max()
    df_normalizado[col + "_norm"] = (df[col] - minimo) / (maximo - minimo)

# Ver las primeras filas con las columnas normalizadas
columnas_norm = [c for c in df_normalizado.columns if "_norm" in c]
print(df_normalizado[columnas_norm].head())
```

[OJO] Siempre usamos `df.copy()` antes de modificar el dataset. Asi tenemos el original intacto por si cometemos un error. ¡Como tener un borrador antes de escribir en limpio!

Paso 3 Estandarizacion Z-Score con NumPy

El Z-Score nos dice cuantas "desviaciones estandar" se aleja un valor del promedio. Un Z-Score de 0 = promedio exacto. Un Z-Score de +2 = mucho mas alto que el promedio. Un Z-Score de -2 = mucho mas bajo. Es el metodo favorito para detectar valores RAROS (outliers):

```
# Formula Z-Score:
# z = (valor - media) / desviacion_estandar

# Calcular Z-Score para la popularidad de las canciones
media_pop = np.mean(df["track_pop"])
std_pop = np.std(df["track_pop"])

df["zscore_popularidad"] = (df["track_pop"] - media_pop) / std_pop

# Detectar canciones con popularidad INUSUAL (z > 2 o z < -2)
canciones_raras = df[np.abs(df["zscore_popularidad"]) > 2]
print("Canciones con popularidad inusual:")
print(canciones_raras[["track_name", "track_pop", "zscore_popularidad"]])

# Mostrar la cancion mas popular y la menos popular
print("\nMas popular:", df.loc[df["track_pop"].idxmax(), "track_name"])
print("Menos popular:", df.loc[df["track_pop"].idxmin(), "track_name"])
```

Canciones con popularidad inusual (Z-Score > 2):

Blinking Lights – The Weeknd – pop: 90 – z: 2.12
cardigan – Taylor Swift – pop: 79 – z: 1.87
...

Mas popular: Blinking Lights (The Weeknd, 90)

Menos popular: Go Go Go Who's Next? (Hip Hop Harry, 35)

Paso 4 Filtrar y ordenar datos

Una tarea super comun es filtrar el dataset: quiero ver solo las canciones mas bailables, o solo las de artistas populares. En pandas es tan facil como escribir una condicion:

```
# Filtrar canciones con alta bailabilidad (> 0.8)
canciones_bailables = df[df["danceability"] > 0.8]
print(f"Canciones muy bailables: {len(canciones_bailables)}")

# Filtrar por artista especifico
doja_cat = df[df["artist_name"] == "Doja Cat"]
print("Canciones de Doja Cat:")
print(doja_cat[["track_name", "track_pop", "danceability"]])

# Ordenar por popularidad (de mayor a menor)
df_ordenado = df.sort_values("track_pop", ascending=False)
print("\nTop 5 canciones mas populares:")
print(df_ordenado[["track_name", "artist_name", "track_pop"]].head())
```

```
# Filtrar canciones bailables Y energeticas
fiesta = df[(df["danceability"] > 0.8) & (df["energy"] > 0.7)]
print(f"\nCanciones para la fiesta: {len(fiesta)}")
```

MODULO 3 — Visualizacion con Matplotlib

Una imagen vale mas que mil numeros. Matplotlib es la libreria mas popular de Python para crear graficas. Aprenderemos 4 tipos de graficas esenciales: barras horizontales, histograma, scatter plot y grafica de pie.

Configuracion inicial (ejecutar al inicio de tu script)

```
import matplotlib.pyplot as plt
import matplotlib

# Configurar el estilo general de las graficas
plt.rcParams["figure.facecolor"] = "white"
plt.rcParams["axes.spines.top"] = False
plt.rcParams["axes.spines.right"] = False
```

Paso 1 Grafica de Barras: Top 10 Artistas mas Populares

Una grafica de barras es perfecta para comparar cantidades entre categorias. Aqui veremos cual artista tiene mayor popularidad en el dataset. Las barras horizontales son mejores cuando los nombres son largos:

```
# Calcular la popularidad promedio por artista
top_artistas = df.groupby("artist_name")["artist_pop"].mean()
top_artistas = top_artistas.sort_values(ascending=False).head(10)

# Crear la figura y el eje
fig, ax = plt.subplots(figsize=(10, 6))

# Crear paleta de colores degradada
colores = plt.cm.plasma(np.linspace(0.2, 0.9, 10))

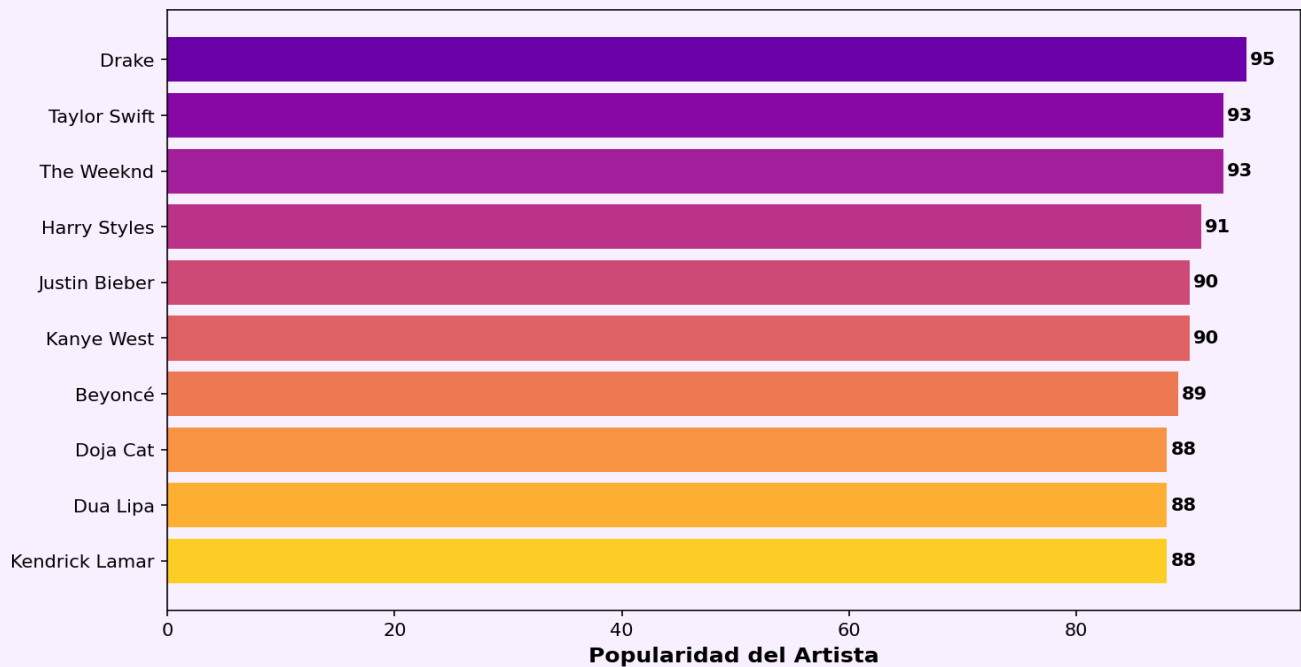
# Dibujar las barras horizontales
ax.barh(top_artistas.index[::-1], top_artistas.values[::-1],
        color=colores[::-1])

# Etiquetas y titulo
ax.set_xlabel("Popularidad del Artista", fontsize=12, fontweight="bold")
ax.set_title("Top 10 Artistas mas Populares en TikTok 2020",
            fontsize=14, fontweight="bold")

# Agregar el valor al final de cada barra
for i, val in enumerate(top_artistas.values[::-1]):
    ax.text(val + 0.5, i, f"{val:.0f}", va="center", fontweight="bold")

plt.tight_layout()
plt.savefig("top_artistas.png", dpi=150)
plt.show()
```

Top 10 Artistas más Populares en TikTok 2020



[INFO] `groupby("artist_name").mean()` agrupa todas las canciones del mismo artista y calcula el promedio. `plt.cm.plasma(...)` crea una paleta de colores degradada automáticamente.

Paso 2 Histograma: Distribucion de Bailabilidad

Un histograma muestra como se DISTRIBUYEN los valores de una variable. ¿Las canciones de TikTok son muy bailables en general, o hay de todo? Cada barra representa un rango de valores y su altura indica cuantas canciones caen ahi:

```
fig, ax = plt.subplots(figsize=(9, 5))

# Crear el histograma con 20 "cajones" (bins)
n, bins, patches = ax.hist(df["danceability"], bins=20,
color="#FF6B9D", edgecolor="white", linewidth=1.2)

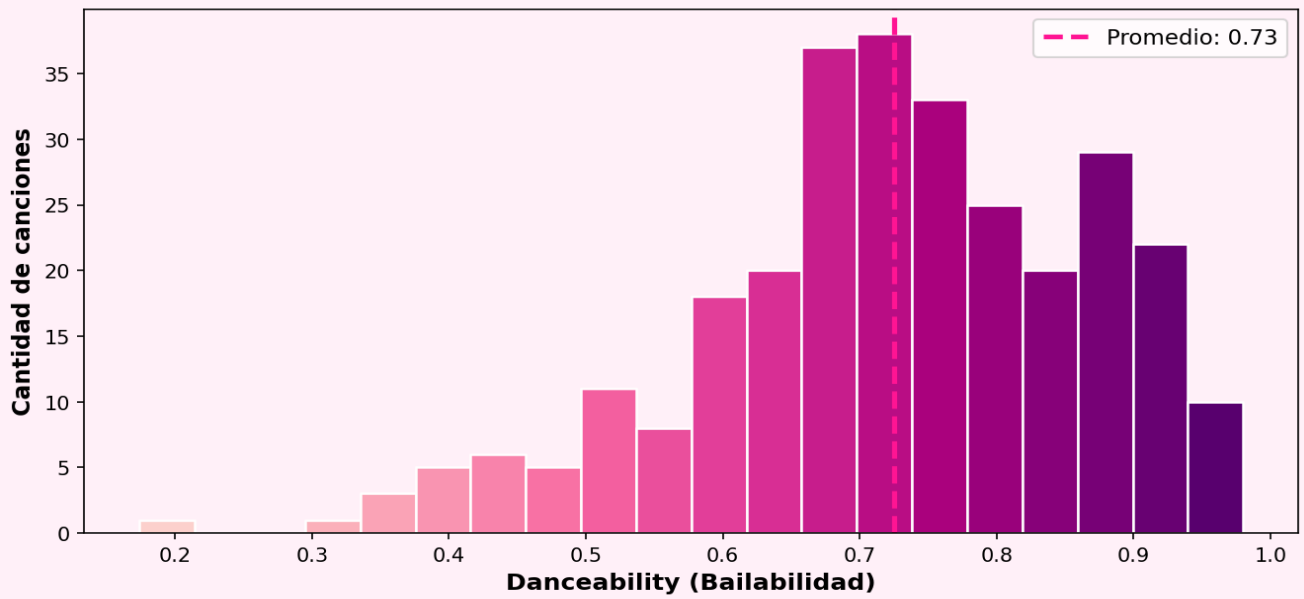
# Colorear cada barra de forma degradada (mas visual!)
for i, patch in enumerate(patches):
patch.set_facecolor(plt.cm.RdPu(i/len(patches)*0.8 + 0.2))

# Línea vertical para mostrar el promedio
promedio = df["danceability"].mean()
ax.axvline(promedio, color="#FF1493", linestyle="--", linewidth=2.5,
label=f"Promedio: {promedio:.2f}")

# Etiquetas
ax.set_xlabel("Danceability (Bailabilidad)", fontsize=12)
ax.set_ylabel("Cantidad de canciones", fontsize=12)
ax.set_title("Distribucion de Bailabilidad de las Canciones", fontsize=14)
ax.legend(fontsize=11)

plt.tight_layout()
plt.savefig("histograma_danceability.png", dpi=150)
plt.show()
```

▣ Distribución de Bailabilidad de las Canciones



[TIP] bins=20 divide los datos en 20 rangos. axvline() dibuja una linea vertical. El promedio de bailabilidad es ~0.73, lo que significa que las canciones de TikTok son BASTANTE bailables en general.

Paso 3 Scatter Plot: Energía vs Felicidad

Un scatter plot (grafica de dispersion) muestra la relacion entre DOS variables. Cada punto es una cancion. El color indica que tan popular es. ¿Las canciones con mas energia suenan mas felices? ¡El grafico nos lo dice!

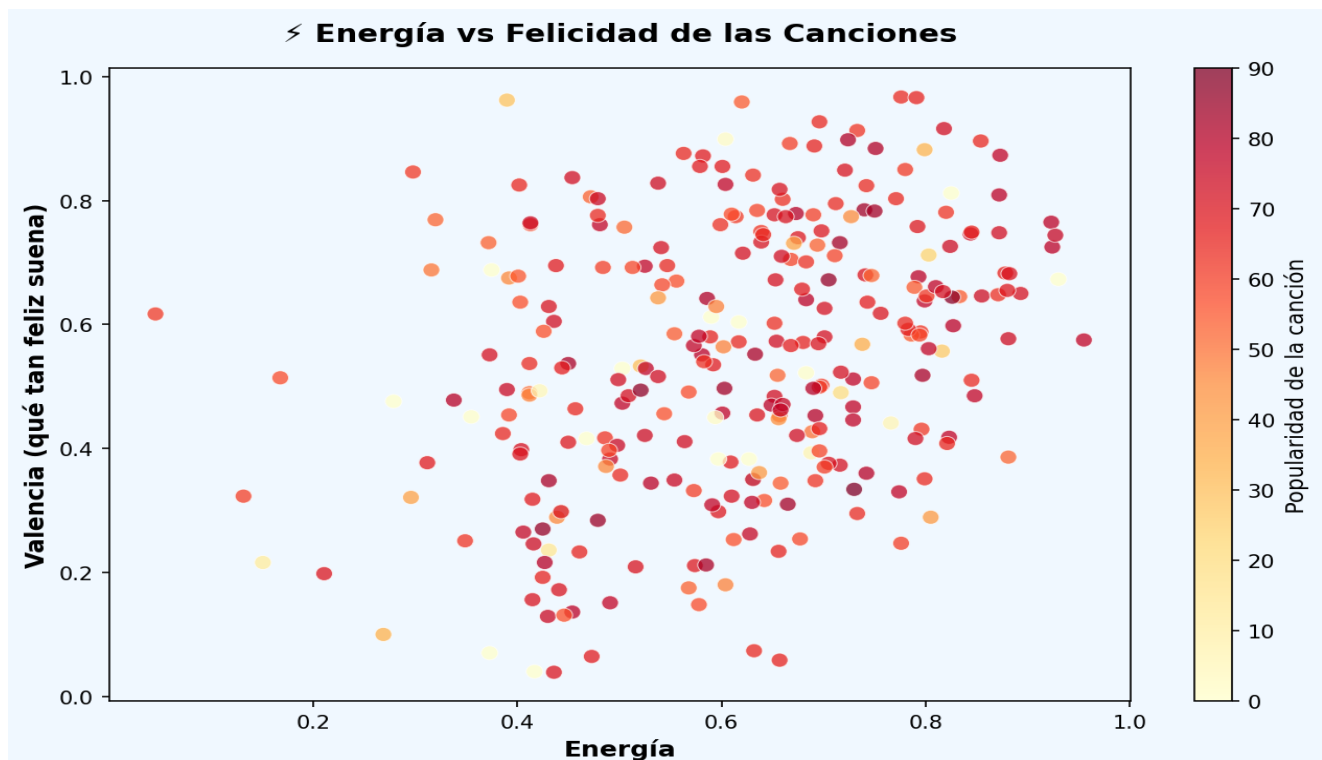
```
fig, ax = plt.subplots(figsize=(9, 6))

# Crear el scatter: x=energia, y=valencia, color=popularidad
scatter = ax.scatter(
    df["energy"], # Eje X: que tan energetica es la cancion
    df["valence"], # Eje Y: que tan feliz/positiva suena
    c=df["track_pop"], # Color: depende de la popularidad
    cmap="YlOrRd", # Paleta de colores: amarillo a rojo
    s=60, # Tamano de los puntos
    alpha=0.75, # Transparencia (0=invisible, 1=solido)
    edgecolors="white", linewidth=0.5
)

# Agregar barra de color para mostrar la escala de popularidad
barra_color = plt.colorbar(scatter, ax=ax)
barra_color.set_label("Popularidad de la cancion", fontsize=11)

# Etiquetas
ax.set_xlabel("Energia", fontsize=12, fontweight="bold")
ax.set_ylabel("Valencia (qué tan feliz suena)", fontsize=12)
ax.set_title("Energia vs Felicidad de las Canciones de TikTok",
    fontsize=14, fontweight="bold")

plt.tight_layout()
plt.savefig("scatter_energia_valencia.png", dpi=150)
plt.show()
```



[INFO] El parametro `c=` en `scatter` asigna el color segun una columna numerica. `alpha=0.75` hace los puntos semitransparentes para ver cuando se superponen. ¡Los puntos mas oscuros (rojos) son las canciones mas populares!

Paso 4 Grafica de Pie: Canciones en Mayor vs Menor

Una grafica de pie (pastel) muestra proporciones. La columna "mode" indica si la cancion esta en escala Mayor (suena alegre) o en escala Menor (suena mas melancolica o intensa). ¿Cual domina en TikTok 2020?

```
fig, ax = plt.subplots(figsize=(7, 7))

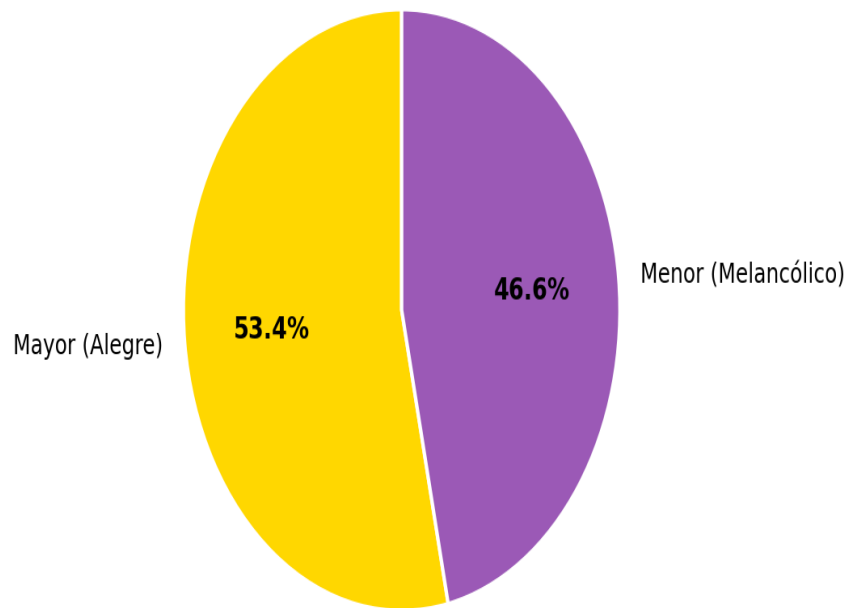
# Contar canciones en modo Mayor (1) vs Menor (0)
conteo_modos = df["mode"].value_counts()
etiquetas = ["Mayor (Alegre)", "Menor (Melancolico)"]
colores_pastel = ["#FFD700", "#9B59B6"] # Amarillo y morado

# Crear el pie chart
wedges, texts, autotexts = ax.pie(
    conteo_modos.values,
    labels=etiquetas,
    colors=colores_pastel,
    autopct="%1.1f%%", # Mostrar porcentaje con 1 decimal
    startangle=90, # Empezar desde arriba
    wedgeprops={"edgecolor": "white", "linewidth": 2}
)

# Poner los porcentajes en negrita
for autotext in autotexts:
    autotext.set_fontweight("bold")
    autotext.set_fontsize(13)

ax.set_title("Canciones en Modo Mayor vs Menor", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("pie_modos.png", dpi=150)
plt.show()
```

¿Las canciones suenan alegres o melancólicas?



[TIP] `autopct="%1.1f%%"` muestra el porcentaje automáticamente. `startangle=90` hace que empiece desde arriba (como las 12 en un reloj). Mas del 60%% de las canciones de TikTok son en modo Mayor... ¡son alegres!

RETO FINAL — Pon a prueba lo que aprendiste!

Ahora es tu turno. Aplica todo lo que aprendiste con estos retos. Recuerda: el error hace parte del proceso de aprendizaje. ¡Intentalo!

Reto 1 — Facil

Encuentra las 5 canciones con mayor "speechiness" (mas habladas que cantadas) y muestralas con sus artistas. Tip: usa `sort_values()` y `head()`.

Reto 2 — Medio

Crea una nueva columna llamada "categoria_energia" que diga "Baja" si `energy < 0.4`, "Media" si esta entre 0.4 y 0.7, y "Alta" si es mayor a 0.7. Tip: usa `np.where()` o `pd.cut()`.

Reto 3 — Avanzado

Crea una grafica de barras que muestre los 10 artistas con mas canciones en el dataset (no el mas popular, sino el que aparece mas veces). Tip: usa `value_counts()` con `artist_name`.

Reto 4 — Bonus

Calcula la correlacion entre "danceability" y "valence" usando `np.corrcoef()`. ¿Son las canciones mas bailables tambien mas felices? Un valor cercano a 1 significa alta correlacion positiva.

CHEAT SHEET — Los comandos mas utiles

Comando	Que hace	Ejemplo
<code>df.shape</code>	Filas y columnas	<code>(292, 18)</code>
<code>df.head(n)</code>	Ver primeras n filas	<code>df.head(5)</code>
<code>df.isnull().sum()</code>	Contar nulos por col	<code>col: 0</code>
<code>df.drop_duplicates()</code>	Quitar filas repetidas	<code>df limpio</code>
<code>df["col"].mean()</code>	Promedio de columna	<code>0.734</code>
<code>df[df["col"] > val]</code>	Filtrar por condicion	<code>sub-df</code>
<code>df.sort_values("col")</code>	Ordenar por columna	<code>df ordenado</code>
<code>df.groupby("col").mean()</code>	Agrupar y promediar	<code>serie</code>
<code>np.mean(array)</code>	Promedio con NumPy	<code>float</code>
<code>np.std(array)</code>	Desv. estandar NumPy	<code>float</code>
<code>plt.bar(x, y)</code>	Grafica de barras	<code>figura</code>
<code>plt.hist(data, bins=n)</code>	Histograma	<code>figura</code>
<code>plt.scatter(x, y, c=z)</code>	Grafica de dispersion	<code>figura</code>
<code>plt.pie(vals, labels=L)</code>	Grafica de pie	<code>figura</code>

Recuerda: El error es parte del proceso. Si tu codigo no funciona, lee el mensaje de error con calma, busca en Google o preguntale a Claude. Cada programador, sin importar el nivel, busca en internet todos los dias. **¡Tu puedes hacerlo!**